

Relazione – Stabilizzazione di un pendolo inverso con RL su STM32F446RE

Oct 9, 2026 · @Salvatore

Abstract

Una politica Soft Actor-Critic (SAC) stabilizza il pendolo inverso su carrello e gira su una STM32F446RE in 904 μ s per passo, su un periodo di controllo di 20 ms. La rete (6-128-128-1) riceve posizione, angolo e velocità stimate e comanda la frequenza firmata degli impulsi STEP del motore passo-passo. È addestrata in simulazione MuJoCo con rumore di misura (5-10 mm, 0,5-1°) e disturbi sui parametri, poi convertita con X-CUBE-AI. Nella prova hardware-in-the-loop sulla scheda reale il pendolo resta stabile in 5/5 episodi da 20 s e 10/10 da 60 s, con angolo rms 0,64° e posizione rms 2,6 cm; le uscite della scheda coincidono con quelle del PC entro 1e-6.

Introduzione

L'obiettivo è mantenere in equilibrio verticale un pendolo montato su un carrello mosso da un motore passo-passo, usando una rete neurale addestrata con reinforcement learning ed eseguita interamente su un microcontrollore STM32F446RE. Il pendolo inverso è instabile in anello aperto e la sua dinamica non lineare lo rende un banco di prova classico per il controllo.

Rispetto a un regolatore LQR, che in una prima fase del progetto è stato usato e poi rimosso, la politica RL impara direttamente dalla simulazione e tollera rumore e variazioni dei parametri senza un modello linearizzato esplicito. La sfida embedded è eseguire rete e stimatore dello stato entro il periodo di controllo di 20 ms, con risultati identici a quelli del PC.

Gli obiettivi del progetto sono:

1. modellare il sistema carrello-pendolo con attuatore a passo e simularlo in MuJoCo;
2. addestrare una politica SAC robusta a rumore di misura, vibrazioni e incertezze sui parametri;
3. portare rete e stimatore sulla STM32F446RE con X-CUBE-AI e verificarli in hardware-in-the-loop (HIL);
4. preparare la prova sul banco reale.

La relazione descrive nell'ordine il modello, la formulazione RL, l'addestramento, l'implementazione sul microcontrollore e i risultati, chiudendo con i limiti noti e gli sviluppi futuri. Lo swing-up (portare il pendolo dal basso all'alto) non è incluso: tutti gli episodi partono vicino alla verticale.

Modello del sistema carrello-pendolo

Il sistema è un carrello su guida lineare, trascinato da cinghia dentata e motore passo-passo, con un'asta incernierata libera di ruotare nel piano verticale. Lo stato è $(x, \dot{x}, \theta, \omega)$: posizione e velocità del carrello, angolo dell'asta (0 = verticale in alto, π = in basso) e velocità angolare. L'angolo è misurato da un encoder da 2400 conteggi/giro sul perno.

Modello fisico in MuJoCo

Il modello è descritto in `cart_pendolo.xml` e integrato a 2 ms (500 Hz) con integratore `implicitfast`. Le masse sono calcolate da MuJoCo dalle singole geometrie.

Parametro	Valore
Massa traslante del carrello	363 g (piastra, supporti, cuscinetti, ruote, encoder)
Asta	acciaio $\varnothing$ 5 mm, braccio superiore 0,82 m, inferiore 0,16 m
Corsa utile	$\pm 0,462$ m (contatto con i finecorsa)
Smorzamento giunto carrello / pendolo	0,1 / 0,01
Attuatore	velocità del carrello, $k_v = 200$
Passo di integrazione	2 ms

Il motore passo-passo è rappresentato come attuatore di velocità: impone al carrello la velocità corrispondente alla frequenza degli impulsi STEP, con una forza proporzionale all'errore di velocità. Non servono quindi coppia, rigidezza o inerzia del rotore; la dinamica del pendolo resta accoppiata fisicamente al carrello.

Dinamica del pendolo

Con il carrello mosso in velocità, l'accelerazione del carrello a entra come ingresso nell'equazione del pendolo:

$$\ddot{\theta} = k_g \sin \theta - k_a \cos \theta, \quad a_{\text{carrello}} - k_d \omega$$

con $k_g = 17,13 \text{ s}^{-2} (\text{m} \cdot \text{l} \cdot \text{g} / \text{J})$, $k_a = 1,746 \text{ rad/m}$ e $k_d = 0,350 \text{ s}^{-1}$, ricavati dal modello MuJoCo con `stima.coefficienti_da_modello`. La stessa equazione è usata dallo stimatore dello stato sul firmware.

Attuazione: dagli impulsi STEP alla velocità

Il motore ha 200 passi/giro con microstepping 32 (6400 impulsi/giro); la puleggia ha 48 denti con cinghia a passo 2 mm, quindi ogni impulso sposta il carrello di 0,015 mm.

$$v_{\text{carrello}} = f_{\text{STEP}} \cdot \frac{p_{\text{cinghia}}}{z_{\text{puleggia}} \cdot n_{\text{passi}} \cdot \mu_{\text{step}}}$$

La frequenza massima di 32 000 Hz corrisponde a 0,48 m/s. La frequenza applicata segue quella richiesta con una rampa di $1,6 \cdot 10^6 \text{ Hz/s}$ (24 m/s^2 sul riferimento), aggiornata ogni 2 ms: da 0 a 32 000 Hz servono 20 ms.

Ambiente di simulazione e formulazione RL

Il problema è formulato come ambiente Gymnasium (`ambiente.py`) con fisica a 500 Hz e decisioni della rete a 50 Hz: a ogni azione corrispondono 10 passi fisici. Un episodio dura 20 s (1000 decisioni) e parte con il pendolo vicino alla verticale: x uniforme in $\pm 10 \text{ cm}$, θ in $\pm 0,05 \text{ rad}$, ω in $\pm 0,02 \text{ rad/s}$.

Azione

La rete emette un solo valore $u \in [-1, 1]$: il segno sceglie il verso (pin DIR), il modulo la frequenza degli impulsi STEP. Con zona morta d e frequenza minima f_{min} :

$$f = \text{sign}(u) \left[f_{\text{min}} + \frac{|u| - d}{1 - d} (f_{\text{max}} - f_{\text{min}}) \right], \quad |u| > d$$

Nella configurazione addestrata $d = 0$, $f_{\text{min}} = 3200 \text{ Hz}$ e $f_{\text{max}} = 32\,000 \text{ Hz}$. La frequenza richiesta passa poi per la rampa descritta sopra prima di diventare velocità del carrello.

Osservazioni

La rete riceve sei ingressi normalizzati:

#	Ingresso	Normalizzazione
1	posizione del carrello x	/ 0,462 m (limite di corsa)
2	$\sin \theta$	—
3	$\cos \theta$	—
4	velocità del carrello	/ 0,48 m/s
5	velocità angolare ω	/ 10 rad/s
6	frequenza applicata dopo la rampa	/ 32 000 Hz

L'uso di $\sin \theta$ e $\cos \theta$ evita la discontinuità dell'angolo a $\pm\pi$. Il sesto ingresso è uno stato interno noto al controllore e dà alla rete la memoria del comando ancora in corso di rampa.

Ricompensa

La ricompensa per passo premia il pendolo in alto, il carrello al centro e una bassa velocità angolare, e penalizza comandi grandi e bruschi:

$$r = \frac{1 + \cos\theta}{2} \left(0,6 + 0,4 \cdot e^{-(x/0,15)^2} \right) \left(0,5 + 0,5 \cdot e^{-0,1 \cdot \omega^2} \right) - 0,005 \cdot u^2 - 0,02 \cdot (u - u_{\text{prec}})^2$$

La forma moltiplicativa fa sì che centratura e quiete continuo solo con il pendolo in alto. La scala di 15 cm sulla posizione è stata ristretta dopo che, con un termine più largo, il carrello derivava fino al finecorsa nelle prove da 60 s. Il contatto con un finecorsa ($|x| \geq 0,462$ m) termina l'episodio con una penalità di -5 ; il massimo teorico per episodio è circa 1000.

Criterio di successo

Un episodio è riuscito se resta stabile per tutta la durata: $|\theta| < 12^\circ$, $|x|$ entro la corsa, $|\omega| < 1$ rad/s e $|\dot{x}| < 0,75$ m/s in ogni istante.

Rumore di misura e stimatore dello stato

Sul banco x e θ sono misurati con rumore, e derivarli ogni 20 ms rende le velocità inutilizzabili: 7,5 mm di rumore su x danno circa 0,53 m/s di rumore sulla velocità, più della velocità massima del carrello. Per questo la rete non riceve velocità derivate ma stimate da un osservatore (`stima.py`) che:

1. ogni 2 ms predice θ e ω con l'equazione del pendolo e la x con la velocità applicata dalla rampa;

2. ogni 20 ms corregge la predizione con le misure: $\theta \pm 0,2 \cdot r$, $\omega \pm 1,0 \cdot r$ (r = errore angolare), $x \pm 0,05 \cdot (x_{\text{mis}} - x)$.

Disturbi e randomizzazione del dominio

Per rendere la politica robusta, il profilo `disturbi.toml` (intensità 0,25) aggiunge a ogni episodio:

Disturbo	Intervallo
Rumore di posizione (1σ per lettura)	5–10 mm
Rumore dell'encoder (1σ , prima della quantizzazione a 2400 cpr)	0,5–1°
Offset angolare	fino a $\pm 0,125^\circ$
Masse e inerzie	$\pm 2,5\%$
Smorzamento dei giunti	$\pm 6,25\%$
Modo oscillante cinghia (forza sul carrello)	8–18 Hz, fino a 0,05 N
Modo oscillante motore (forza sul carrello)	20–40 Hz, fino a 0,025 N
Modo oscillante asta (coppia sul pendolo)	3–10 Hz, fino a 0,00075 N·m

I modi oscillanti sono oscillatori smorzati (ζ 0,08–0,25) eccitati dall'accelerazione del riferimento e da rumore correlato su 40 ms. Sono disturbi equivalenti, non un modello strutturale identificato sul banco.

Addestramento

La politica è addestrata con Soft Actor-Critic (SAC) di Stable-Baselines3, un algoritmo off-policy per azioni continue che massimizza ricompensa ed entropia della politica. L'attore finale è una rete $6 \rightarrow 128 \text{ ReLU} \rightarrow 128 \text{ ReLU} \rightarrow 1 \text{ tanh}$; i due critici hanno la stessa struttura nascosta.

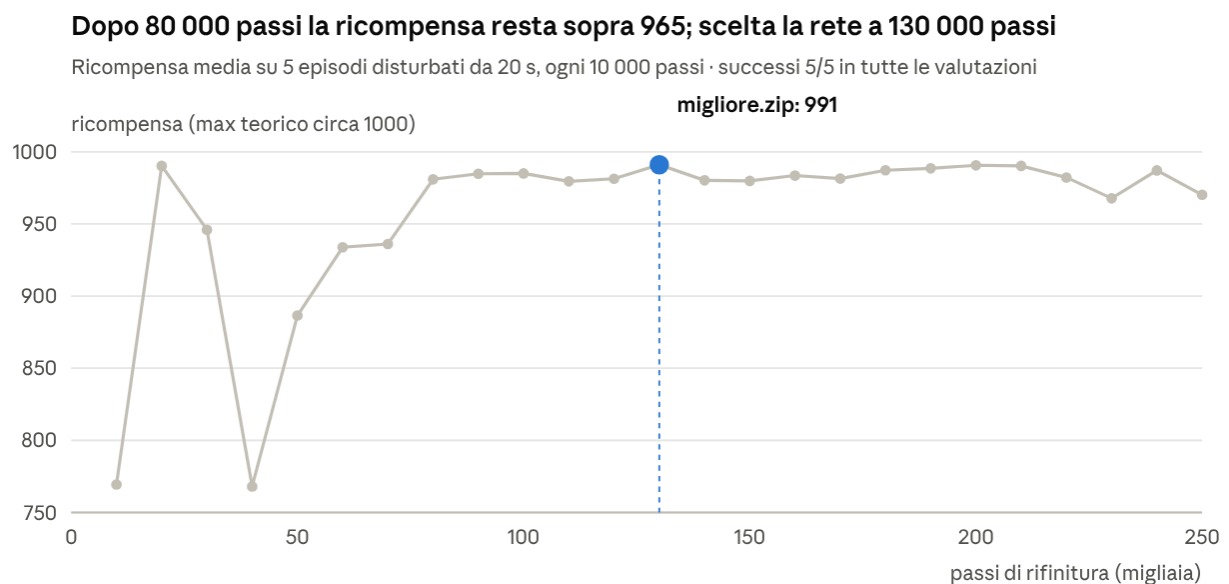
Iperparametro	Valore
Learning rate	$3 \cdot 10^{-4}$
Replay buffer	300 000 transizioni
Passi prima dell'apprendimento	3000
Batch	128

Iperparametro	Valore
γ (sconto)	0,99
τ (aggiornamento target)	0,005
Frequenza di training	1 aggiornamento ogni 4 passi
Coefficiente di entropia	automatico, iniziale 0,1
Architettura	[128, 128]

L'addestramento si è svolto in due fasi:

1. 400 000 passi da zero nell'ambiente con disturbi e rumore;
2. 250 000 passi di rifinitura con la ricompensa attuale (centratura a 15 cm e penalità sulle variazioni del comando), ripartendo dai pesi della fase 1 con replay buffer vuoto.

Ogni 10 000 passi la rete è valutata su 5 stati iniziali fissi, sia nell'ambiente nominale sia in quello disturbato. Il checkpoint salvato come `migliore.zip` è quello che massimizza la peggiore durata stabile fra le due suite, a parità la ricompensa media. La rete scelta (130 000 passi della rifinitura) ha 5/5 successi in entrambe le suite, ricompensa media 991 disturbata e 994 nominale. Le firme SHA-256 di XML, ambiente, stepper e stimatore sono salvate nel checkpoint, così una rete non può essere caricata con un ambiente diverso da quello di addestramento.



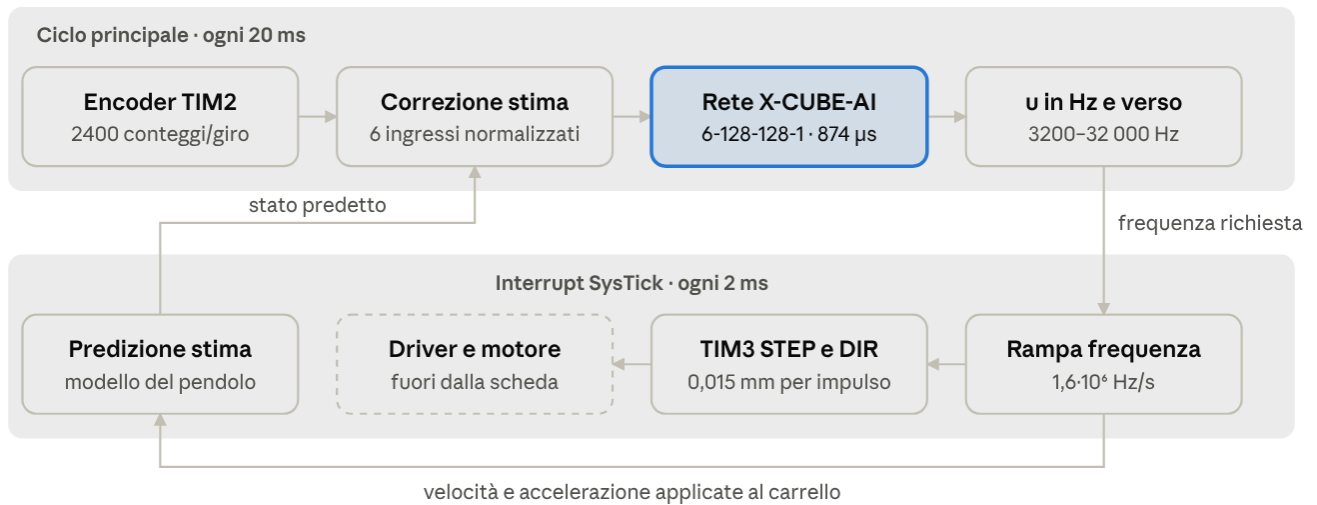
modelli_disturbi/progresso.csv · 25 valutazioni, 10 000–250 000 passi

I due cali iniziali (769 a 10 000 e 768 a 40 000 passi) cadono nella fase iniziale della rifinitura, con il replay buffer ancora quasi vuoto; da 80 000 passi in poi la rete resta stabile e la ricompensa oscilla fra 968 e 991.

Implementazione embedded su STM32F446RE

Rete e stimatore girano su una Nucleo-F446RE (Cortex-M4F, 180 MHz, FPU a singola precisione) nel progetto STM32CubeIDE `Prova_controllo`. Il firmware completo occupa 97,6 kB di flash ed è compilato con arm-none-eabi-gcc 13.2 senza warning.

Stimatore e rete usano 904 μ s dei 20 ms del ciclo



catena di controllo sul microcontrollore · due livelli temporali

Ogni 2 ms la rampa aggiorna la frequenza STEP e lo stimatore predice lo stato; ogni 20 ms la misura dell'encoder corregge la stima e la rete calcola il nuovo comando.

Dalla rete addestrata al codice C

1. L'attore di `modelli_disturbi/migliore.zip` è esportato in ONNX (opset 13, float32): $\text{Gemm} \rightarrow \text{ReLU} \rightarrow \text{Gemm} \rightarrow \text{ReLU} \rightarrow \text{Gemm} \rightarrow \text{Tanh}$.
2. L'esportazione è validata su 5100 ingressi contro PyTorch: errore massimo $8,6 \cdot 10^{-6}$, medio $1,1 \cdot 10^{-7}$.
3. ST Edge AI Core 2.1 (X-CUBE-AI) genera il codice C per stm32f4, senza compressione.
4. Dopo ogni nuovo addestramento `aggiorna_pesi_xcubeai.py` riscrive i pesi in `network_data_params.c`, senza rigenerare il progetto.

Risorsa della rete	Valore
Parametri	17 537 (68,5 KiB in flash, float32)
Operazioni MAC per inferenza	17 803
RAM di attivazione	1024 B
Tempo di <code>ai_network_run</code> misurato	874 μ s

Risorsa della rete	Valore
Stimatore + rete per ciclo	904 μ s (4,5% del periodo di 20 ms)

Periferiche e tempi

Funzione	Periferica	Periodo
Lettura encoder del pendolo (2400 cpr)	TIM2 in modalità encoder	ogni 20 ms
Generazione impulsi STEP	TIM3 PWM, ARR e CCR aggiornati al volo	continuo
Rampa di frequenza e predizione dello stimatore	SysTick	2 ms
Correzione dello stimatore, ingressi e inferenza (<code>Rete_Process</code>)	ciclo principale	20 ms
Verso del motore	GPIO DIR	a ogni cambio di segno
Avvio/arresto del controllo	pulsante B1 (antirimbazzo 50 ms)	—

La posizione del carrello non è misurata: è ricostruita contando gli impulsi STEP emessi rispetto al centro trovato con l'homing, e la sua velocità è quella programmata nel timer. Lo stimatore in C (`stima.c` , costanti in `stima.h` uguali a `stimatore.toml`) riproduce `stima.py` : su 20 sequenze da 60 s la differenza massima è $1,4 \cdot 10^{-6}$.

Sequenza di funzionamento

1. All'accensione il firmware inizializza rete e periferiche, poi attende che l'encoder resti fermo per 1 s con il pendolo appeso e fissa $\theta = \pi$.
2. L'homing porta il carrello sui finecorsa e lo riporta al centro, che diventa $x = 0$.
3. Con il pendolo portato a mano in alto, B1 avvia il controllo; una seconda pressione ferma subito il PWM.
4. Ogni 20 ms il ciclo corregge la stima, prepara i sei ingressi, esegue la rete e converte u in frequenza STEP e verso, applicati con la rampa.
5. Ingressi non finiti, superamento del limite di corsa o mancato rispetto della scadenza di 40 ms fermano il motore e registrano un codice d'errore.

Modalità hardware-in-the-loop

Con `HIL_MODE 1` la scheda non comanda il motore: riceve via USART2 (115 200 baud) le misure simulate dal PC, esegue stimatore e rete e restituisce l'azione insieme al tempo di inferenza. Lo script `hil.py` chiude l'anello con la simulazione MuJoCo e, in parallelo, calcola stimatore e rete in Python per confrontarli passo per passo con la scheda.

Risultati sperimentali e validazione

La politica ha stabilizzato il pendolo in tutti i 205 episodi di prova, dalla simulazione pura fino all'HIL con la scheda reale, senza mai raggiungere un finecorsa. La validazione è stata condotta per gradi, avvicinandosi ogni volta di più all'hardware.

Prova	Episodi	Successi	Note
Simulazione, semi 2000–2019, 20 s, nominale	20	20/20	ricompensa media 995
Simulazione, semi 2000–2019, 20 s, disturbi 0,25	20	20/20	ricompensa media 988
Simulazione lunga, 60 s: nominale, disturbi 0,25 e 1, rumore massimo, spinte ± 1 N per 100 ms	100	100/100	angolo max $4,5^\circ$, carrello entro 15 cm
Catena del firmware simulata (stima.c + pesi X-CUBE-AI, float32), 60 s	60	60/60	angolo max $6,6^\circ$
HIL su Nucleo-F446RE, 20 s	5	5/5	angolo rms $0,64^\circ$, x rms 2,6 cm
HIL su Nucleo-F446RE, 60 s	10	10/10	angolo rms $0,64^\circ$, x rms 2,6 cm

Prova hardware-in-the-loop

Nella prova HIL del 9 ottobre 2026 la scheda ha chiuso l'anello su 15 episodi (35 000 passi di controllo) con disturbi a intensità 0,25. Gli ingressi calcolati dalla scheda differiscono da quelli di `stima.py` al massimo di $4,5 \cdot 10^{-7}$ e l'azione di X-CUBE-AI da quella di PyTorch al massimo di $5,8 \cdot 10^{-7}$: differenze di arrotondamento float32. Non ci sono state ritrasmissioni sulla seriale.

Seme (60 s)	Angolo max (°)	Angolo rms (°)	x max (cm)	x rms (cm)
7000	3,00	0,69	7,1	2,4
7001	2,48	0,56	9,4	2,1
7002	1,98	0,64	5,9	2,5
7003	2,65	0,77	6,4	2,6
7004	1,63	0,49	6,3	2,8
7005	2,23	0,64	7,3	2,7
7006	1,51	0,44	7,1	3,4
7007	3,26	0,84	7,7	2,7
7008	4,54	0,72	14,8	2,8
7009	1,52	0,46	4,0	1,3

Il tempo di `ai_network_run` è 874 μ s, costante in tutti i passi (media 873,75 μ s); stimatore e rete insieme richiedono al massimo 904 μ s, cioè il 4,5% del periodo di controllo. Resta quindi ampio margine per una rete più grande o un periodo più breve.

Comportamento del comando

L'analisi dei passi HIL mostra che la rete lavora con comandi piccoli ($|u|$ medio 0,08) e cambia verso nel 33% dei passi. Poiché la frequenza minima in movimento è 3200 Hz, ogni inversione corrisponde a un salto di velocità di $\pm 0,048$ m/s. In simulazione questo non degrada la stabilizzazione, ma sul motore reale ogni inversione richiede il cambio del pin DIR e può causare vibrazioni o passi persi: è il punto principale da verificare sul banco.

Conclusioni e sviluppi futuri

Il progetto dimostra che una politica SAC addestrata solo in simulazione può stabilizzare il pendolo inverso ed essere eseguita su una STM32F446RE con meno del 5% del periodo di controllo e risultati identici al PC entro 10^{-6} . Due scelte sono state decisive: comandare direttamente la frequenza degli impulsi STEP, così che rete e firmware parlino la stessa lingua del driver, e fornire alla rete velocità stimate da un osservatore basato sul modello invece che derivate dalle misure rumorose.

I limiti attuali sono:

- la validazione si ferma all'HIL: motore, driver, encoder e meccanica reali non sono ancora stati provati;

- la posizione del carrello deriva dal conteggio degli impulsi comandati e non rileva eventuali passi persi;
- i disturbi e i parametri di massa sono ipotesi ragionevoli, non identificati sul banco;
- le frequenti inversioni di verso del comando potrebbero causare vibrazioni sul motore reale.

Gli sviluppi previsti sono:

1. prova sul banco reale (`HIL_MODE 0`), verificando verso dell'angolo e segni dei comandi;
2. riduzione delle inversioni del comando con una zona morta, una frequenza minima più bassa o una penalità nella ricompensa;
3. identificazione sperimentale di masse, attriti e frequenze di vibrazione, per restringere i disturbi a valori misurati;
4. estensione allo swing-up, portando il pendolo dalla posizione di riposo alla verticale.